

```
Administrator: Command Prompt
C:\Users\grnib\Downloads>python -m pip show torch
Name: torch
Version: 2.1.1.0
Summary: Tensors and Dynamic neural networks in Python with strong GPU acceleration
Home-page: https://pytorch.org
Author:
Author-email: PyTorch Team <packages@pytorch.org>
License: BSD-3-Clause
Location: C:\Users\grnib\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages
Requires: filelock, fsspec, Jinja2, networkx, setuptools, sympy, typing-extensions
Required-by: torchvision

C:\Users\grnib\Downloads>py ai_security_capstone.py
100.0%
100.0%
100.0%
100.0%

=== Training Baseline ===
Epoch 1, Loss: 166.7929
Epoch 2, Loss: 44.3729
Accuracy: 98.27%

=== Running FGSM Attack ===
FGSM Accuracy with epsilon=0.0: 98.27%
FGSM Accuracy with epsilon=0.05: 93.76%
FGSM Accuracy with epsilon=0.1: 78.25%
FGSM Accuracy with epsilon=0.15: 42.58%
FGSM Accuracy with epsilon=0.2: 12.03%
FGSM Accuracy with epsilon=0.25: 2.77%
FGSM Accuracy with epsilon=0.3: 1.29%
Saved: sample_adversarial_examples.png

=== Adversarial Training Defense ===
Defense Epoch 1, Loss: 638.6614
Defense Epoch 2, Loss: 301.6140
Accuracy: 95.96%
FGSM Accuracy with epsilon=0.0: 95.96%
FGSM Accuracy with epsilon=0.05: 94.56%
FGSM Accuracy with epsilon=0.1: 93.55%
FGSM Accuracy with epsilon=0.15: 93.10%
FGSM Accuracy with epsilon=0.2: 93.49%
FGSM Accuracy with epsilon=0.25: 94.08%
FGSM Accuracy with epsilon=0.3: 93.39%
```

```
Administrator: Command Prompt
FGSM Accuracy with epsilon=0.25: 2.77%
FGSM Accuracy with epsilon=0.3: 1.29%
Saved: sample_adversarial_examples.png

=== Adversarial Training Defense ===
Defense Epoch 1, Loss: 638.6614
Defense Epoch 2, Loss: 301.6140
Accuracy: 95.96%
FGSM Accuracy with epsilon=0.0: 95.96%
FGSM Accuracy with epsilon=0.05: 94.56%
FGSM Accuracy with epsilon=0.1: 93.55%
FGSM Accuracy with epsilon=0.15: 93.10%
FGSM Accuracy with epsilon=0.2: 93.49%
FGSM Accuracy with epsilon=0.25: 94.08%
FGSM Accuracy with epsilon=0.3: 93.39%

=== Saving Graphs ===
Saved: accuracy_vs_epsilon.png
Saved: defense_comparison.png

===== RESULTS =====
Baseline clean accuracy: 98.27%
Baseline under attack (e=0.25): 2.77%
Defended clean accuracy: 95.96%
Defended under attack (e=0.25): 94.08%
Improvement under attack: +91.31%
=====
```

Screenshots of the model being trained, results shown in screenshots.

PYTHON CODE (some lines were changed from the original given code to avoid the model from crashing, as well as the implementation of graph visual lines to retrieve images for the visual results of the model being trained with its attack and defense)

ai_security_capstone_sample.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

1. Setup

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
transform = transforms.Compose([
    transforms.ToTensor()
])
```

```
train_data = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)
```

```
test_data = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)
```

```
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=1000, shuffle=False)
```

2. Simple CNN Model

```

-----
class SimpleCNN(nn.Module):
    def init(self):
        super(SimpleCNN, self).init()
        self.conv1 = nn.Conv2d(1, 16, 3, 1)
        self.conv2 = nn.Conv2d(16, 32, 3, 1)
        self.fc1 = nn.Linear(32 * 24 * 24, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SimpleCNN().to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

```

3. Train Baseline Model

```

-----
def train(model, loader, epochs=2):
    model.train()

    for epoch in range(epochs):
        total_loss = 0

        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)

            loss.backward()
            optimizer.step()

        total_loss += loss.item()

    print(f'Epoch {epoch + 1}, Loss: {total_loss:.4f}')

```

----- 4. Evaluate Model -----

```
def evaluate(model, loader):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            predictions = outputs.argmax(dim=1)

            correct += (predictions == labels).sum().item()
            total += labels.size(0)

    accuracy = correct / total
    print(f"Accuracy: {accuracy * 100:.2f}%")
    return accuracy
```

----- 5. FGSM Adversarial Attack -----

```
def fgsm_attack(image, epsilon, gradient):
    perturbation = epsilon * gradient.sign()
    adversarial_image = image + perturbation
    adversarial_image = torch.clamp(adversarial_image, 0, 1)
    return adversarial_image

def evaluate_fgsm(model, loader, epsilon=0.25):
    model.eval()
    correct = 0
    total = 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        images.requires_grad = True

        outputs = model(images)
        loss = criterion(outputs, labels)

        model.zero_grad()
```

```

loss.backward()

gradient = images.grad.data
adversarial_images = fgsm_attack(images, epsilon, gradient)

adversarial_outputs = model(adversarial_images)
predictions = adversarial_outputs.argmax(dim=1)

correct += (predictions == labels).sum().item()
total += labels.size(0)

accuracy = correct / total
print(f"FGSM Accuracy with epsilon={epsilon}: {accuracy * 100:.2f}%")
return accuracy

```

----- 6. Simple Defense: Adversarial Training -----

```

def adversarial_train(model, loader, epochs=2, epsilon=0.25):
    model.train()

    for epoch in range(epochs):
        total_loss = 0

        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            images.requires_grad = True

            outputs = model(images)
            loss = criterion(outputs, labels)

            model.zero_grad()
            loss.backward()

            gradient = images.grad.data
            adversarial_images = fgsm_attack(images, epsilon, gradient)

            optimizer.zero_grad()
            defended_outputs = model(adversarial_images.detach())
            defended_loss = criterion(defended_outputs, labels)

            defended_loss.backward()
            optimizer.step()

```

```
total_loss += defended_loss.item()
```

```
print(f"Defense Epoch {epoch + 1}, Loss: {total
```